



Acciones masivas: historial y reversión segura

02 de octubre de 2026

Qué se ejecutó sobre muchos registros a la vez, quién lo hizo, qué cambió en cada uno, y cómo deshacerlo **sin pisar lo que alguien tocó después**.

Dónde: Sistema → Acciones masivas. Solo superadministrador.

1. Qué problema resuelve

Una importación de 800 contratos mal armada, un corte masivo lanzado con la lista equivocada, una corrida de facturación de un período que no tocaba. Hasta ahora cada uno de esos procesos dejaba, como mucho, una línea en la auditoría: servía para saber **qué pasó**, no para **deshacerlo**. Lo que faltaba no era la bitácora —que ya existe— sino guardar, registro por registro, **cómo estaba antes**.

2. Las dos piezas

`mass_actions` — la operación: tipo, estado, usuario, sucursal, contadores, un resumen propio de cada proceso (el archivo, el período, el umbral) y el enlace con su reversión.

`mass_action_items` — un registro afectado: qué se tocó, su etiqueta legible (`EGP000123`), el estado, y `before/after` **con solo los campos que cambiaron**.

`after` no es decoración: es contra lo que se compara el estado **actual** antes de revertir. Si no coincide, alguien lo cambió después y no se toca.

Las tablas que ya existían —`billing_runs`, `contract_cutoffs`, `ont_import_runs`— **siguen donde estaban** con sus datos propios y sus pantallas. La acción masiva las enlaza; no las sustituye.

3. Estados

Estado	Qué significa
<code>procesando</code>	Todavía corriendo. No se revierte: no se sabe qué va a dejar
<code>completada</code>	Terminó sin errores
<code>completada_con_errores</code>	Terminó, con registros que fallaron

Estado	Qué significa
fallida	No llegó a hacer nada útil
revirtiendo	Reversión en curso
revertida	Deshecha del todo
reversion_parcial	Se deshizo lo que se pudo; quedaron conflictos

Y por registro: `ok`, `omitido`, `error`, `revertido`, `conflicto`, `no_reversible`.

Omitido no es error. Omitido es una decisión del sistema —ese contrato no debía dos meses—: la operación hizo lo correcto. Error es que algo se rompió. Juntarlos haría que una corrida sana pareciera un desastre.

4. Qué se registra y qué se revierte

Operación	Reversión	Qué hace exactamente
Corte masivo por cartera	Sí	Devuelve el estado anterior y restablece el servicio (ONT y PPPoE)
Corte masivo de PPPoE	Sí	Rehabilita el secret en el Mikrotik
Cobro múltiple	Sí	Reversa los pagos y su movimiento de caja — solo con la caja abierta
Movimiento de almacén	Sí	Registra el movimiento contrario ; el original no se borra
Importación de clientes	Sí	Borra los contratos creados y lo que cuelga
Importación de ONTs	Sí	Borra las ONT del sistema ; la OLT no se toca
Corrida de facturación	Sí	Anula las facturas anulables — no las borra

Las siete se registran siempre, se reviertan o no.

4.1 Las cuatro que piden explicación

Cobro múltiple. Se reversa cada pago del lote: la factura vuelve a quedar con su saldo y se quita el movimiento de la caja. **El estado del contrato no se toca** aunque el pago lo hubiera reactivado — cortarle el servicio a alguien por un cobro mal registrado sería castigarlo por un error ajeno, y la mora se recalcula sola en la siguiente corrida.

Un pago del lote también se puede reversar SUELTO desde Facturación → Pagos: cae solo ese y los demás quedan como están. Los dos caminos usan el mismo servicio, `PaymentReverser`, así que tienen los mismos frenos y dejan la misma huella.

Movimiento de almacén. Se deshace con el movimiento **contrario**, no borrando: una entrada con una salida, un traslado con el traslado inverso. Los dos quedan a la vista. Borrar el original dejaría un almacén que cuadra y un historial que no explica por qué.

Importación de ONTs. Borra **solo las filas**. La importación LEYÓ la OLT y copió lo que encontró: no configuró nada. Mandarle un borrado al equipo sería hacer algo que la importación nunca hizo, y dejar sin servicio a clientes que llevan meses navegando por deshacer una lectura. Las ONT siguen autorizadas: volver a importar las trae de nuevo.

Corrida de facturación. Anula, nunca borra. Una factura gastó un consecutivo de un rango autorizado que no se recupera, y el hueco hay que justificarlo ante la DIAN.

5. Reversión segura: las cuatro reglas

5.1 No sobrescribe cambios posteriores

Antes de tocar un registro se comprueba que siga **exactamente** como lo dejó la acción.

Esperado (lo que dejó el corte):	Suspendido	
Estado actual:	Suspendido	→ se revierte
Esperado:	Suspendido	
Estado actual:	Retirado	→ CONFLICTO, no se toca

El conflicto se muestra con las tres versiones y su motivo. **Nunca se sobrescribe en silencio.**

El conflicto es del **intento**, no del registro: el ítem conserva su estado `ok` —la acción sí lo cambió— y guarda el motivo por el que hoy no se pudo deshacer. Así se puede reintentar cuando se resuelva lo que estorbaba, y la acción no se da por revertida sin estarlo.

5.2 No se revierte dos veces

La acción se bloquea (`lockForUpdate`) y se comprueba dentro de la transacción: dos pestañas abiertas o un doble clic llegan a la vez, y sin eso las dos pasarían. Además cada ítem lleva la fecha en que se revirtió — un trabajo que se reintenta no vuelve a tocarlo.

5.3 No lo carga todo en memoria

Los ítems se recorren con `chunkById` en lotes de 200: esto funciona igual con 800 que con 50.000.

5.4 Un registro no tumba a los demás

Cada ítem va en su **propia transacción**. El que falla queda marcado con su motivo y los otros ochocientos siguen.

6. Lo que la reversión se NIEGA a hacer

Hechos consumados que una fila borrada no puede deshacer. Cada uno deja el registro en **conflicto** con su motivo, para que lo decida una persona.

6.1 Dinero y documentos

Freno	Por qué
Factura con pagos	Entró dinero y está en el cuadre de una caja. Borrarla deja ese dinero sin documento que lo soporte
Factura validada por la DIAN	Existe en sus registros con su CUF. Borrarla aquí no la borra allá: los dos lados dejan de coincidir
Notas crédito o débito	Son documentos con su propio consecutivo, emitidos y entregados

6.2 La caja cerrada

Un cobro múltiple solo se reversa **mientras su caja siga abierta**. Si ya se cerró, el arqueado de ese turno se hizo con ese dinero dentro: quitarlo después lo descuadra para siempre y sin nada que lo explique. Con la caja cerrada se resuelve con las figuras que sí dejan rastro de los dos lados — una nota, un egreso o un ajuste.

6.3 El material que ya no está

Un movimiento de almacén no se revierte si las existencias ya no están donde las dejó. Un equipo que entró y después se instaló en casa de un cliente no se puede «desentrar»: la unidad que habría que quitar ya no está en ese almacén, y restarla igual lo dejaría en negativo.

6.4 El trabajo de otro

Una ONT importada que alguien vinculó después a un contrato no se borra: ese vínculo es trabajo posterior y borrarla se lo llevaría por delante.

7. Cómo se revierte, paso a paso

1. **Sistema → Acciones masivas**. Filtrar por tipo, estado, usuario o fecha.
2. **Ver** la acción: información general, resumen, y los registros afectados con su antes/después.
3. **Revertir acción**. Lleva a la confirmación, que **antes de pedir nada** dice cuántos se pueden deshacer, cuántos están en conflicto y por qué, y qué va a pasar exactamente.
4. Escribir **REVERTIR**. No es un «¿está seguro?»: obliga a leer.
5. Al terminar se abre la **acción de reversión**, con su resultado.

Las dos acciones quedan enlazadas en los dos sentidos: desde la original se ve quién la deshizo, y desde la reversión cuál deshizo.

8. Añadir un proceso masivo nuevo

Tres pasos, y nada más se entera:

1. **Un caso en** `MassActionType` con su etiqueta e icono.

2. El proceso avisa, sin cambiar su firma ni lo que devuelve:

```
$accion = $recorder->abrir(
    MassActionType::LoQueSea,
    'Descripción de lo que se va a hacer',
    summary: ['lo' => 'propio del proceso'],
    source: $registroPropio,      // opcional
);

foreach (...) {
    $recorder->registrar($accion, $modelo, 'EGP000123',
        antes: ['status' => 'Activo'],
        despues: ['status' => 'Suspendido'],
    );
}

$recorder->cerrar($accion);
```

`antes` y `despues` llevan **solo los campos que cambian**. Con 50.000 ítems, copiar el registro entero multiplica la base para guardar cuarenta columnas que nadie va a mirar.

3. Si se puede deshacer, una clase que implemente `RevierteUnaAccionMasiva` y una línea en `MassActionRegistry`. La interfaz son dos métodos separados a propósito:

- `revisar()` responde «¿puedo tocar esto?» **sin tocar nada**.
- `revertir()` lo deshace, y solo se le llama si lo primero dijo que sí.

Juntarlos es como nacen las reversiones que pisan el trabajo de otro: basta un `return` mal puesto para que la comprobación se salte y el cambio no.

Lo que no esté en el registro **no se revierte**, y esa es la respuesta correcta: mejor que el historial lo diga a que un botón prometa algo que no puede cumplir.

9. Seguridad

Reservado al **superadministrador**, y por **rol activo de la sesión** —no por un permiso marcapable en el módulo de roles—. Es el mismo criterio que la trazabilidad y las copias de seguridad, por la misma razón: desde aquí se deshacen cortes, se anulan facturas y se borran

contratos.

Dos puertas, y protegen cosas distintas:

- el middleware `superadmin` en la ruta → la pantalla;
- `MassActionPolicy` → la acción concreta sobre una acción concreta, y si esa acción se puede revertir siquiera.

Escribir la URL a mano no sirve: hay prueba de ello.

10. Dónde está en el código

Qué	Dónde
Registrar una operación	<code>app/MassActions/MassActionRecorder.php</code>
Deshacerla	<code>app/MassActions/MassActionReverter.php</code>
Qué estrategia deshace cada tipo	<code>app/MassActions/MassActionRegistry.php</code>
El contrato de una estrategia	<code>app/MassActions/RevierteUnaAccionMasiva.php</code>
Las estrategias	<code>app/MassActions/Reversiones/</code>
La aritmética del inventario, compartida	<code>app/Services/InventoryMover.php</code>
Estados y tipos	<code>app/MassActions/Enums/</code>
Modelos	<code>app/Models/MassAction.php</code> , <code>MassActionItem.php</code>
Pantalla	<code>app/Http/Controllers/MassActionController.php</code>
Autorización	<code>app/Policies/MassActionPolicy.php</code>

Las siete estrategias están en `app/MassActions/Reversiones/`: `RevertirCorteDeContratos`, `RevertirCortePppoe`, `RevertirCobroMultiple`, `RevertirMovimientoDeAlmacen`, `RevertirImportacionDeClientes`, `RevertirImportacionDeOnts` y `AnularCorridaDeFacturacion`.

InventoryMover salió del controlador de movimientos cuando hubo que deshacerlos: la reversión necesita exactamente la misma aritmética —seriales, promedio ponderado, el costo que viaja en los traslados— y dos copias es como acaban diciendo cosas distintas del mismo almacén. El controlador conserva su método y delega.

Pruebas: `tests/Feature/System/MassActionsTest.php` (registro, reversión, conflictos, doble reversión, seguridad) y `MassActionReversalGuardsTest.php` (lo que la reversión debe negarse a hacer: factura con pago, validada por la DIAN, contrato importado con pagos o con notas, y la caja cerrada — con sus contrapuntos, porque si solo se prueba lo que se niega, un sistema que no revierte nada pasa todas las pruebas).